

Angular 22 + Playwright + TDD

Vitest · TestBed · storageState · wcag2aa · Red-Green-Refactor

Vitest entry

```
import { vi } from "vitest";

describe("CourseStore Integration", () => {
  // TestBed + provideHttpClient()
});
```

Lock the TMS frontend surface: components, stores, real-browser, accessibility

Zoneless testing mindset

- Angular 22 is zoneless by default — signals, explicit change detection, Vitest-class runners
- Reduce blind fakeAsync/tick everywhere; prefer async/await, fixture.whenStable() for async work and signal effects
- Flakiness usually comes from wrong waits, not from deleting zone.js alone — verify exact APIs against your Angular minor's testing docs

Setting up Vitest (Angular 22)

course.store.spec.ts (trimmed)

```
// course.store.spec.ts – Vitest + TestBed (example scaffold)
```

```
import { vi } from "vitest";
```

```
describe("CourseStore Integration", () => {
```

```
  beforeEach(() => {
```

```
    TestBed.configureTestingModule({
```

```
      providers: [CourseStore, provideHttpClient()],
```

```
    });
```

```
  });
```

```
});
```

- Vitest is the default unit-test runner for new Angular projects from recent major versions
- Karma → Vitest is a separate programme decision — follow [Migrating to Vitest on angular.dev](#) and your CLI output for the exact version you ship

Testing signals: TestBed.tick()

CourseHubService (trimmed)

```
// course-hub.service.spec.ts
it("should run effects when signal changes", () => {

  const service = TestBed.inject(CourseHubService);

  service.status.set("Critical");

  // Forces the Signal scheduler to run effects
  TestBed.tick();

  expect(service.log()).toContain("Status updated to Critical");

});
```

- Reads are synchronous; effect() work is scheduled — TestBed.tick() drains the queue so assertions see the log side effect
- Angular deprecated the older TestBed.flushEffects() name in favour of TestBed.tick() — functionally identical today

Mocking HTTP (common trap)

HttpTestingController (trimmed)

```
// course-service.spec.ts
const httpMock = TestBed.inject(HttpTestingController);

const service = TestBed.inject(CourseService);

service.getAll().subscribe((courses) => {

  expect(courses).toHaveLength(1);

  expect(courses[0].title).toBe("Regional Bootcamp");

});

const req = httpMock.expectOne(
  (r) => r.url.includes("/courses") &&
    r.params.get("page") === "1" &&
    r.params.get("pageSize") === "50",
);

req.flush({ items: [...], totalCount: 1 });

httpMock.verify();
```

- Do not mock HttpClient — register provideHttpClientTesting() in TestBed providers for specs that hit the wire
- Use HttpTestingController to assert URLs, flush bodies, and verify — keeps TMS /api/courses reads honest at the boundary

The "Big Thread" E2E test

- Unit tests check the bricks; E2E tests check the bridge — does the full TMS stack cooperate?
- Scenario: instructor logs in, opens a course, posts a grade; an admin (or authorized role) sees the update on the dashboard without manual refresh
- Goal: answer whether the entire TMS system works together for real roles and real routes

Authentication setup (shared storage state)

auth.setup.ts (trimmed)

```
// auth.setup.ts – use the same labels/roles your login page exposes

import { test as setup, expect } from "@playwright/test";

setup("authenticate as Abeba", async ({ page }) => {

  await page.goto("/login");

  await page.getByLabel(/email|username/i).fill("admin@tms.local");

  await page.getByLabel("Password").fill(process.env.TMS_ADMIN_PASS!);

  await page.getByRole("button", { name: "Sign In" }).click();

  await expect(page.getByRole("heading", { name: /dashboard/i })).toBeVisible();
  await page.context().storageState({ path: "playwright/.auth/user.json" });

});
```

- Setup project once, save storageState, reuse for authenticated flows — full form login every test is slow and brittle
- M10 TMS default: HttpOnly auth cookie (session or JWT in cookie — not localStorage); storageState persists those cookies
- Mutating flows still need real XSRF behaviour (synchronizer cookie + header) when M10 antiforgery is enabled

Testing API failure (Playwright route)

resilience.spec.ts (trimmed)

```
// resilience.spec.ts - Mocking API Failure
test("shows 'Server error' banner when enrollments API returns 500", async ({ page }) => {
  // Intercept and Force a 500 Error
  // Wildcard matches both V1 and V2 controllers from M7 - survives a future migration.

  await page.route(/\api\v\d+\.\d+\enrollments(\..*?)?$/, (route) => {
    route.fulfill({
      status: 500,
      contentType: "application/problem+json",
      body: JSON.stringify({
        type: "https://tms.local/errors/server_

        title: "Server error",

        status: 500,

        detail: "An unexpected error occurred.",
      }),
    });
  });

  await page.goto("/dashboard");
  // Mirrors the M7 GlobalExceptionHandler.ProblemDetails.Title = "Server error".
  // Asserting on role+name (not plain text) means a CSS refactor won't break this.
  await expect(page.getByRole("alert", { name: /server error/i })).toBeVisible();
});
```

- Solid E2E includes unhappy paths — not only sunny-day clicks; route.fulfill keeps the browser honest while the API is forced offline

Interview angle

- Explain whenStable vs fakeAsync for the Angular version on your CV project
- Show TestBed.tick() when an assertion depends on a signal effect() (Angular deprecated the older flushEffects() name)
- Name when you would reach for Vitest browser mode vs JSDOM (real DOM, styles, focus)
- Prefer Playwright locators and built-in auto-wait over arbitrary sleeps
- Describe auth setup + storageState: reuse HttpOnly cookies from a real login (M10), not hand-pasting JWTs
- Include at least one failure story (5xx, timeout, empty state) beside happy-path flows
- Describe AxeBuilder runs on critical routes (login, course list, grade flow) and how you triage violations vs noise

Common mistakes — Session 2

Mistake	Symptom	Fix
Using fakeAsync/tick without zone.js	Tests pass but assertions silently skip	Use async/await + fixture.whenStable()
Direct property assignment for signal inputs	Component does not re-render in test	fixture.componentRef.setInput('name', value)
Referencing fixture in a service-only test	fixture is undefined	Use TestBed.inject(Service) for services, not fixture
Using Jasmine spyOn instead of Vitest vi.spyOn	Spy does not work or is not defined	Import vi from vitest and use vi.spyOn()
Forgetting TestBed.tick() before assertion	Signal effect has not fired yet	Call TestBed.tick() after the signal change (was flushEffects(); deprecated)
Missing XSRF header in mutating tests	API returns 400 (Antiforgery token)	Setup must perform a real login to get the XSRF-TOKEN cookie
Hardcoding passwords in setup files	Passwords leak in version control	Use process.env.TMS_ADMIN_PASS!
Committing storageState to Git	Auth session theft via GitHub	Add playwright/.auth/ to .gitignore
Testing only sunny-day paths	Test suite is green, production is down	Add 5xx, timeout, and empty-state scenarios using page.route
CSS selectors for primary actions	Tests break on every styling refactor	Use getByRole or getByLabel instead
waitForTimeout() for loading states	Flaky tests on slow networks/CI	Use await expect(locator).toBeVisible()
Running axe without withTags	Too many violations, most irrelevant	Use .withTags(['wcag2aa']) for focused scanning

Module 12 complete — TMS safety net

Unit, integration, component, E2E — full stack coverage from Rules to Routes.